Midterm 2 Review Sheet

CS 5090 · Software Engineering with Generative AI · review segment Monday, November 16 (second half of class) · exam Wednesday, November 18, in class

The exam at a glance

- **When:** Wednesday, November 18, during class (50 minutes). Arrive on time.
- **Rules:** closed book, closed notes, no phones, laptops, or AI tools. No calculator needed; the pass@k formula and the few values you need are printed on the exam.
- **Format:** mostly conceptual and mostly circle-the-answer. 25 multiple-choice questions (2 points each), 20 true/false statements (1 point each), two matching tables (10 points each: write the term from a word bank next to its description), one short-answer question (10 points, three or four sentences), and one bonus question (10 points) that is the hardest part of the exam. No work-it-out problems; the one number on the exam uses values printed on the paper.
- **Cumulative, split by Midterm 1:** about **70 points on everything since Midterm 1**: the Week 7–8 basics (RAG, the agent loop, evaluation and contamination) and Module 2 (Machine Learning in Production, the eight lectures from "the model is a small box" through responsible engineering); and about **30 points on the Midterm 1 material** (Weeks 1–5: tokens, attention, pretraining and scaling, post-training, pass@k). Not on the exam: the ⚙ sessions (specs, assuring generated code, productivity).
- **Best preparation:** do the Module 2 worksheet without looking at the answers, then check yourself; reread the Key-takeaways slide of each lecture; redo the Midterm 1 review sheet's practice problems for the Weeks 1–5 part. Most exam questions are close to a worksheet question or a practice problem below. The practice problems ask you to write more than the exam will; that is the right way to study for a multiple-choice exam.

Module 2 · What you should be able to do

L1 · From models to systems

- Explain why the model is "the small box": name the components around it in the transcription example and in a coding assistant (serving, context, telemetry, fallback).
- State the five properties that make ML systems different (no specification, mistakes are certain, the world moves, data is code, entanglement) and give the consequence of each.
- Say why most ML product failures are systems failures, and what the V-model's three characteristics (decomposition, boundaries, consistent V&V) are an answer to.

L2 · When to use ML, goals and measurement

- Apply the five conditions for using ML (there is a pattern too complex to write by hand; you have representative data; mistakes are tolerable or recoverable; the task keeps changing; the value beats the cost) and the mistake-severity grid to a scenario.
- Order the four layers of goals (organisational → product/leading indicator → user outcome → model quality) and explain Booking.com's finding that a better model is not a better product.
- Ask the three questions of any metric (valid, reliable, affordable) and say what Goodhart's law predicts.

L3 · Requirements and planning for mistakes

- Separate the world, the machine, and the assumption that connects them; tell apart system, user, environment, and component requirements.
- Explain why a model's residual error rate never reaches zero and list the six patterns for designing for failure (undo, lower forcefulness, guardrail, human in the loop, redundancy, graceful degradation).
- Do a harm → hazard → cause → control analysis, and read a fault tree: it starts from the harm you are accountable for and works backwards to causes and controls (AND/OR, where a mitigation goes).

L4 · Architecture, deployment, operations

- Name the five quality attributes (latency, cost, privacy, availability, updatability) and use them to choose server-side, on-device, or hybrid.
- Explain training/serving skew: why it is silent and why the fix is one shared feature-encoding path.

- Describe the pipeline with an evaluate gate; SLOs and observability; what MLOps adds to DevOps (data and models as versioned artefacts, monitoring that fires with no deployment).

L5 · Quality assurance at four levels

- Place a defect at the model, data, pipeline, or system level.
- Explain why a model can be validated but not verified; why one accuracy number hides things; and the three behavioural test types (minimum function, invariance, directional) plus "known failures".
- Distinguish data drift from concept drift; give an example of a silent pipeline defect (label leakage); say what system-level tests cover (safeguards, failure paths, operations).

L6 · Testing in production, versioning

- Explain shadow, canary, A/B test, and feature flag, and what each limits.
- State the rules of an A/B test (metric and sample size first, no peeking) and what a natural label is.
- Describe feedback loops and how to break them; list the four things to version (code, data, model, environment) and walk the provenance chain from a prediction back to its data; the first step is logging the model version with every prediction.

L7 · Process, teams, technical debt

- Explain why data-science and engineering processes conflict and what timeboxing resolves; what a T-shaped person is.
- List the five kinds of ML technical debt (glue code, configuration debt, undeclared consumers, entanglement, data dependencies); explain CACE and its consequence for testing (no model change is local, so regression-test the whole system); use the debt quadrants (reckless/prudent × deliberate/inadvertent).
- V-model propositions met in Module 2: PR1/3/4 requirements (L3); PR2 architecture with interfaces and owners (L4); PR5/6 V&V at every level with owners (L5); PR7/8 layered decomposition and documentation approved by owners (L7).

L8 · Responsible engineering

- Name the six facets (fairness, explainability, safety, security and privacy, transparency, accountability) and treat them as design properties.
- Explain the three fairness definitions (anti-classification: do not use the protected attribute; independence: equal positive rates across groups; separation: equal error rates across groups) and why independence and separation cannot both hold; global vs local explanations and why a plausible explanation can be wrong.
- Distinguish reliability, safety, and security (a system can be reliable and still unsafe); name the three ML-specific attacks (poisoning, evasion, extraction/leakage); state the four accountability questions (disclosed? explained? appealable? accountable?); say who owns merged generated code (the human who merges it).

Before Midterm 1 · What to review (about 30 points)

- **Tokens and attention:** what a BPE tokenizer does (subword units learned from frequent character sequences; a larger vocabulary gives shorter token sequences); queries, keys, values; why scores are divided by $\sqrt{d_k}$ before softmax (so they do not grow with the key dimension and saturate the softmax); what the causal mask does (no attention to later positions).
- **Pretraining and scaling:** next-token prediction and cross-entropy loss (−ln p of the correct next token, lower when the model gives that token more probability); the Chinchilla rule (about 20 training tokens per parameter; for a fixed compute budget, grow parameters and tokens together); what a loss curve tells you.
- **Post-training:** SFT (demonstrations: prompts paired with good responses), RLHF (a reward model from preferences, then RL), DPO (preferences directly, no reward model), LoRA (frozen base, a small trainable low-rank update, not every weight); what reward hacking is.
- **pass@k:** from n samples and c passes, pass@k = 1 − C(n−c, k)/C(n, k); pass@1 = c/n. The exam prints the formula and the C(n, k) values.

Weeks 7–8 · What to review (counted in the 70 points, with Module 2)

- **Evaluation:** how pass@k is measured (n samples per problem; pass@1 at a low temperature, large k at a higher one; report n, k and temperature); why similarity (BLEU) is not correctness; what contamination is (benchmark problems in the training data inflate the score) and one way to detect it (time split, perturbation).
- **Retrieval and agents:** the four RAG steps (ingest, retrieve, augment, generate); the agent loop (inspect, plan, act, check, adapt) and why a verifier such as tests matters.

Numbers and lists worth knowing

- Five properties of ML systems · five conditions for using ML · four layers of goals · four kinds of requirements · six failure-design patterns · five quality attributes · three model placements · four QA levels · three behavioural test types · four safe ways to ship · four things to version · five kinds of ML debt · six responsibility facets · three fairness definitions · four accountability questions.
- Booking.com: 23 model improvements, no correlation with conversion. A canary serves about 1% of traffic. Chinchilla: ~20 tokens per parameter. C(n, k) is "n choose k": C(5,2) = 10, C(6,2) = 15, C(3,2) = 3, C(4,2) = 6.

Practice problems

Try these before Wednesday; we go through the answers in Monday's review segment; they are not posted.

P1. A start-up wants a model that reads résumés and automatically rejects applicants, with no human review. Which of the five "when ML is the right tool" conditions fails? Name two failure-design patterns that would make the product acceptable.

P2. For a coding assistant, write one metric at each of the four goal layers. Explain why "acceptance rate" alone is a trap and what the honest version is.

P3. A smart-doorbell model "detects package theft". Write the world event, the assumption, and what the machine actually detects. Give two ways the assumption fails. Then fill a harm → hazard → cause → control row for the harm "an innocent neighbour is reported to the police".

P4. A team trains with features normalised by one script and serves with a reimplementation in another language. Nothing crashes; production accuracy is quietly 4 points lower. Name the defect, say why no test caught it, and give the structural fix. At which of the four QA levels does it live?

P5. Classify each defect by QA level (model, data, pipeline, system): (a) a leaked label gives 99% offline accuracy; (b) the rollback procedure has never been run; (c) accuracy is 95% overall and 50% on inputs longer than 500 tokens; (d) production accuracy drifts down for three months with no deployment.

P6. You are shipping a new code-completion model. Lay out a release in four steps using shadow, canary, expand, and flag-off, and say which metric you watch at the canary step and why. Then name the two things an A/B test must fix before it starts.

P7. A customer asks why they were refused a loan eight months ago. List the four artefacts you must have versioned to answer, walk the provenance chain from the prediction back to the data, and give the five-minute fix that makes the first link possible.

P8. "We retrained on newer data; it is a small change, so we only re-ran the model's unit tests." Explain, using CACE and "undeclared consumers", what can go wrong. Which quadrant of the debt grid is this team in?

P9. A lending model satisfies independence (equal approval rates across groups). A regulator asks whether it also satisfies separation. Explain why it generally cannot satisfy both, and what the team must therefore state. Then say in one sentence each what reliability, safety, and security mean for this system.

P10 (Weeks 1–8). For one problem you draw n = 6 samples and c = 3 pass the tests. Compute pass@1, pass@2, and pass^2 (both of two attempts pass). Which number describes what a one-answer assistant delivers? Then: name the four RAG steps, and say what a verifier does in the agent loop.

P11 (Weeks 1–8). In one or two sentences each: what does the causal mask do; what does the Chinchilla rule say; what signal does DPO learn from and how is it different from RLHF; name one way to detect benchmark contamination.

Explain it in two or three sentences

- Why "the model is a small box" is the thesis of Module 2.
- Why a better model is not a better product (Booking.com).
- Why a model's residual error rate never reaches zero, and what follows for design.
- Why training/serving skew is silent, and the fix.
- Why a model can only be validated, never verified.
- Why offline metrics are not enough, and the safe way to test in production.
- Why "small model change" has no meaning (CACE).
- Why "we made it fair" says nothing by itself.
- Who owns merged AI-generated code, and why that is a professional claim.
- What the V-model's decomposition, boundaries, and V&V are an answer to.